

Hash Table and Search Algorithm Analysis

Fitsum Gelaye

Novemeber 2, 2024

1 Introduction

This document records the results of analyzing different search algorithms (Linear Search, Binary Search, and Hash Table Retrieval) and their performance based on comparisons. The asymptotic runtime of each search method is also recorded.

2 Results

The following table shows the results of comparisons made for 42 randomly selected items from the dataset. The table includes comparisons made for Linear Search, Binary Search, and Hash Table Retrieval for each item.

Item	Linear Search Comparisons	Binary Search Comparisons	Hash Table Comparisons
Item 1	381	8	5
Item 2	458	4	4
...	...	...	...
Item 41	564	8	5
Item 42	108	7	2
Average Comparisons	314.81	8.73	3.21

3 Asymptotic Runtime Analysis

- **Linear Search ($O(n)$):** Linear search requires you to go through the list sequentially until whatever you are searching for is found. Due to the sequential nature of the search, the algorithm has an average time complexity of $O(n)$.
- **Binary Search ($O(\log n)$):** Binary search divides the list in half on each iteration, requiring the list to be sorted. The division aspect gives binary search its $O(\log n)$ complexity.
- **Hash Table Retrieval (Average Case $O(1)$):** The hash table allows for near-instant retrieval on average, as hashing maps items directly to buckets. However, retrieval time may degrade to $O(n)$ in the worst case due to potential collisions and chaining.

4 Code Listing

```
#include <iostream>
#include <fstream>
#include <vector>
#include <list>
#include <string>
#include <cmath>
```

```

#include <iomanip>
#include "Algorithms.h"

const int TABLE_SIZE = 250;
const int LINES_IN_FILE = 666;
const std::string FILE_NAME = "magicitems.txt";

// Hash function
int hashFunction(const std::string& str) {
    int hash = 0;
    for (char ch : str) {
        hash += static_cast<int>(std::toupper(ch));
    }
    return (hash * 1) % TABLE_SIZE;
}

// Hash Table Class with Chaining
class HashTable {
public:
    HashTable() : table(TABLE_SIZE) {}

    void insert(const std::string& key) {
        int index = hashFunction(key);
        table[index].emplace_back(key);
    }

    int retrieve(const std::string& key) {
        int index = hashFunction(key);
        int comparisons = 1; // Initial comparison for accessing the bucket

        for (const auto& item : table[index]) {
            comparisons++;
            if (item == key) return comparisons;
        }
        return comparisons; // Return comparisons even if not found
    }

    void analyzeHashValues(const std::vector<std::string>& searchItems) {
        std::vector<int> bucketCount(TABLE_SIZE, 0);
        int totalComparisons = 0;

        std::cout << "Hash Table Retrieval for Selected Items:\n";

        // Count items in each relevant bucket and track comparisons for retrieval
        for (const auto& item : searchItems) {
            int index = hashFunction(item);
            bucketCount[index]++;
            int comparisons = retrieve(item);
            totalComparisons += comparisons;
            std::cout << "Hash: " << item << ", Comparisons: " << comparisons << "\n";
        }

        // Display bucket usage for the selected items only
    }
};

```

```

    int relevantBuckets = 0;
    std::cout << "\nHash Table Usage for Selected Buckets:\n";
    for (int i = 0; i < TABLE_SIZE; ++i) {
        if (bucketCount[i] > 0) {
            relevantBuckets++;
            std::cout << std::setw(3) << i << " ";
            for (int j = 0; j < bucketCount[i]; ++j) {
                std::cout << "*";
            }
            std::cout << " " << bucketCount[i] << "\n";
        }
    }

    // Calculate and display average load and standard deviation for selected buckets
    double averageLoad = static_cast<double>(totalComparisons) / searchItems.size();
    std::cout << "\nAverage comparisons for selected items: "
        << std::fixed << std::setprecision(2) << averageLoad << "\n";

    double sum = 0;
    for (int count : bucketCount) {
        if (count > 0) {
            double deviation = count - averageLoad;
            sum += deviation * deviation;
        }
    }
    double stdDev = std::sqrt(sum / relevantBuckets);
    std::cout << "Standard Deviation for selected buckets: "
        << std::fixed << std::setprecision(2) << stdDev << "\n";
}

private:
    std::vector<std::list<std::string>> table;
};

int main() {
    // Load magic items from file
    std::ifstream inputFile("magicitems.txt");
    if (!inputFile) {
        std::cerr << "Error: Unable to open file magicitems.txt\n";
        return 1;
    }

    std::vector<std::string> magicItems;
    std::string line;
    while (std::getline(inputFile, line)) {
        magicItems.push_back(line);
    }
    inputFile.close();

    // Sort the array for binary search
    mergeSort(magicItems);

    // Initialize hash table and load it with magic items
    HashTable hashTable;

```

```

    for (const auto& item : magicItems) {
        hashTable.insert(item);
    }

// Step 3: Randomly select 42 items for searching
std::srand(static_cast<unsigned int>(std::time(nullptr)));
std::vector<std::string> searchItems;
for (int i = 0; i < 42; ++i) {
    int randomIndex = std::rand() % magicItems.size();
    searchItems.push_back(magicItems[randomIndex]);
}

// Step 4: Perform linear and binary search, tracking comparisons
int totalLinearComparisons = 0;
int totalBinaryComparisons = 0;

std::cout << "Linear Search Comparisons:\n";
for (const auto& item : searchItems) {
    int linearComp = linearSearch(magicItems, item);
    totalLinearComparisons += linearComp;
    std::cout << "Comparisons for " << item << ": " << linearComp << "\n";
}

std::cout << "Binary Search Comparisons:\n";
for (const auto& item : searchItems) {
    int binaryComp = binarySearch(magicItems, item);
    totalBinaryComparisons += binaryComp;
    std::cout << "Comparisons for " << item << ": " << binaryComp << "\n";
}

// Step 5: Calculate and display averages
double averageLinear = static_cast<double>(totalLinearComparisons) / searchItems.size();
double averageBinary = static_cast<double>(totalBinaryComparisons) / searchItems.size();

std::cout << "Average Linear Search Comparisons: " << averageLinear << "\n";
std::cout << "Average Binary Search Comparisons: " << averageBinary << "\n";

hashTable.analyzeHashValues(searchItems);

return 0;
}

```

5 Code Explanation

- **Lines 10–12:** Defines constants:
 - `TABLE_SIZE`: the size of the hash table (250).
 - `LINES_IN_FILE`: the expected number of lines in the input file.
 - `FILE_NAME`: the name of the file containing the list of items (`"magicitems.txt"`).
- **Lines 15–22:** Defines the `hashFunction` function:

- This function takes a string and computes a hash value by summing the ASCII values of the uppercase version of each character.
- It returns the computed sum modulo `TABLE_SIZE`, ensuring that the hash value is within the bounds of the table.
- **Lines 25–95:** Defines the `HashTable` class with methods to manage a hash table using chaining.
 - **Lines 28–30:** `HashTable()` constructor initializes the hash table as a vector of lists.
 - **Lines 32–36:** `insert()` method takes a key, calculates its hash, and inserts it into the corresponding bucket.
 - **Lines 38–44:** `retrieve()` method:
 - * Takes a key and calculates its hash to access the corresponding bucket.
 - * Goes through the bucket, comparing each item to the key, and returns the number of comparisons made.
 - **Lines 45–85:** `analyzeHashValues()` method:
 - * Takes a vector of items and computes the number of items in each bucket used by the selected items.
 - * Calculates the average comparisons for selected items and displays the load distribution and standard deviation across relevant buckets.
- **Lines 95–108:** `main()` function:
 - Opens the input file and reads each line into the `magicItems` vector.
 - If the file cannot be opened the program terminates.
- **Line 111:** Calls `mergeSort` on `magicItems` to prepare the list for binary search.
- **Lines 114–117:** Initializes a `HashTable` object and loads it with items from `magicItems`.
- **Lines 119–125:** Randomly selects 42 items from `magicItems` for search operations and stores them in `searchItems`.
- **Lines 131–136:** Performs linear search on each item in `searchItems`, stores the total comparisons made, and displays individual comparison counts.
- **Lines 138–143:** Performs binary search on each item in `searchItems`, stores the total comparisons made, and displays individual comparison counts.
- **Lines 145–150:** Calculates and displays the average comparisons for linear and binary searches.
- **Line 153:** Calls `analyzeHashValues()` on the `HashTable` object, passing `searchItems` to analyze hash table retrieval comparisons.

6 Conclusion

The analysis highlights the strengths and weaknesses of each search algorithm. Hash table retrieval offers the best performance on average but can suffer from chaining. Binary search provides excellent performance for sorted data, while linear search has the worst time complexity among the three.